HiveOps Frontend Architecture: Beekeeper App

OVERVIEW

The HiveOps Beekeeper Frontend is a specialized Progressive Web App (PWA) designed to provide beekeepers with real-time, data-driven insights in the field.

Purpose: The frontend serves as the primary interface for translating complex ML-based flowering suitability data into actionable intelligence. It enables beekeepers to identify optimal apiary locations and manage hive health with high efficiency.

User Role: The primary user is the Field Beekeeper. Their goal is to utilize environmental data to make site-selection decisions and maintain precise records of hive status, often in areas with limited connectivity.

Design Philosophy:

- **Mobile-First:** Optimized for one-handed operation on mobile devices.
- **Offline-First:** Critical functionality is maintained without an active internet connection.
- **Data-Driven:** The UI is built around the heatmap visualization to facilitate rapid decision-making.

USER FLOW

The HiveOps experience follows a linear progression from data discovery to management:

1. **Initial Access:** The user opens the PWA; the app checks for cached data and initiates a background sync via `/api/sync`.
2. **Visual Discovery:** The user enters the **Map View**, where the system renders a tile-based heatmap.
3. **suitability Analysis:** The user explores different geographic regions. The ML-generated heatmap highlights areas with high flowering suitability.
4. **Actionable Selection:** Upon identifying a high-scoring tile, the user interacts with the map to view specific ML scores from `/api/score`.
5. **Location Persistence:** The user saves high-potential locations to their **Saved Locations** for future deployment.
6. **Operational Management:** The user moves to **Hive Management** to log data for specific hives currently deployed.

7. **Synchronization:** Any field notes or hive updates made while offline are automatically queued and pushed to the backend once a stable connection is detected.

MAIN SCREENS

Home / Dashboard

- **Purpose:** Provides a high-level snapshot of the beekeeping operation.
- **Features:**
  - Summary statistics (Total Hives, Active Locations).
  - Brief weather summary for current location.
  - Quick-access shortcuts to the Map and Alerts.
- **User Actions:** View system status, navigate to primary modules.

Map (Heatmap View)

- **Purpose:** The core decision-support tool using ML-driven spatial data.
- **Features:**
  - **Tile-Based Heatmap:** Layers fetched from `/api/tile` representing flowering suitability.
  - **ML Scoring Overlay:** Detailed breakdown of suitability factors for specific coordinates.
  - **GPS Integration:** "Find My Location" to see immediate surroundings.
- **User Actions:** Pan/Zoom heatmap, tap tiles for suitability details, save current view as a location.

Hive Management

- **Purpose:** CRUD operations for the beekeeper's physical assets.
- **Features:**
  - Searchable list of hives.
  - Status indicators (Health, Productivity, Last Inspection).
  - Detail views for individual hive history.
- **User Actions:** Create new hives, update hive status, delete decommissioned units.

Saved Locations

- **Purpose:** A curated list of potential or active apiary sites.

- **Features:**
  - List of coordinates/tiles saved from the Map View.
  - Suitability score snapshots at the time of saving.
- **User Actions:** Revisit saved locations on the map, add notes to specific sites.

Alerts / Notifications

- **Purpose:** Critical system communication and risk mitigation.
- **Features:**
  - Risk alerts (e.g., sudden suitability drops, environmental threats).
  - System warnings (e.g., sync errors, low battery).
- **User Actions:** Mark alerts as read, view detailed alert context.

Profile / Settings

- **Purpose:** User account and application configuration.
- **Features:**
  - User profile management.
  - Storage management (clearing cache/offline data).
  - Sync frequency settings.
- **User Actions:** Logout, toggle data-saving modes, update personal info.

NAVIGATION STRUCTURE

The application utilizes a **Bottom Tab Navigation** bar for persistent access to core features:

- **Primary Tabs:** Dashboard, Map, Hives, Saved.
- **Design Rationale:** This structure ensures the user can switch between the heatmap and hive records with a single thumb tap, which is essential for ergonomic field use.
- **Fast Switching:** Screen states are preserved in memory to allow near-instant transitions between the map and management lists.

OFFLINE-FIRST BEHAVIOR

HiveOps is built to survive "dead zones" common in rural apiary sites:

- **Local Persistence:** All hive metadata and saved locations are stored in a local database (IndexedDB).
- **Map Caching:** Map tiles and heatmap data are cached using Service Workers.

- **Sync Logic:**
  - The frontend monitors `navigator.onLine`.
  - Mutations (creating/updating hives) are stored in an "Outbox" while offline.
  - On reconnection, the app calls `/api/sync` to reconcile local changes with the backend.

DESIGN PRINCIPLES

- **Minimalist UI:** High contrast and large touch targets to accommodate use in bright sunlight and while wearing field gear.
- **Field Usability:** Information density is kept low; key metrics are prioritized.
- **Decision-Support Visualization:** The UI flow emphasizes the transition from **ML Data** $\rightarrow$ **Heatmap Tiles** $\rightarrow$ **User Decision**. Logic and visual weight are centered on the heatmap rather than generic charts.

FINAL NOTE

The HiveOps frontend is strictly an implementation of the ML $\rightarrow$ Tiles $\rightarrow$ Heatmap $\rightarrow$ Decision workflow. It provides the visual bridge between the complex scoring logic of the backend and